

포인터 실습

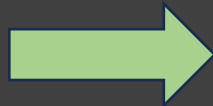
컴퓨터학부 박한영



포인터 선언

- 포인터는 *(asterisk) 연산자를 이용해 선언
- `int *ptr;` 이라고 선언했다고 가정해보자
 - `ptr`은 '`ptr`이 가리키는 것의 주소(address)' 를 의미함
 - `*ptr`은 '`p`가 가리키는 것(value)' 을 의미함
 - `int`는 `*ptr`의 자료형을 나타냄

```
int Var = 3;
int *ptr = &Var;
printf("The value of Var: %d\n", Var);
printf("The address of Var: %#x\n", &Var);
printf("The reference of ptr: %d\n", *ptr);
printf("The value of ptr: %#x", ptr);
```



```
The value of Var: 3
The address of Var: 0x3e1ffa64
The reference of ptr: 3
The value of ptr: 0x3e1ffa64
```



포인터와 문자열

- 문자열을 포인터로 선언하는 경우 상수와 같이 취급함
 - 예: `char *str = "Jaram"` ; , 이 경우 문자열은 코드 영역에 저장
- 문자열의 배열이 필요한 경우, 이중포인터를 사용하여 구현해야 함

```
int n;
scanf("%d", &n);
// char* 를 n개 저장할 배열
char **arr = (char **)malloc(sizeof(char *) * n);
// 각 문자열 공간 할당
for (int i = 0; i < n; i++)
    arr[i] = (char *)malloc(sizeof(char) * 100);
// 문자열 입력
for (int i = 0; i < n; i++)
    scanf("%s", arr[i]);
// 출력
for (int i = 0; i < n; i++)
    printf("%s\n", arr[i]);
// 메모리 해제
for (int i = 0; i < n; i++)
    free(arr[i]);
free(arr);
```



함수의 매개 변수

- 함수의 매개 변수로 포인터를 요구하는 경우, 포인터 변수를 넘겨주거나 변수의 주소값을 넘겨주어야 함.
- C언어에서는 call by reference를 지원하지 않음

```
int main() {  
    int a, b;  
    sum(&a, &b);  
}
```

함수 호출

a, b의 주소가 x, y로 전달

```
int sum(int *x, int *y) {  
    return *x + *y;  
}
```



동적 할당

- 동적할당을 사용하면 프로그램을 실행하는 동안 필요한만큼 공간을 할당 받을 수 있음
- C언어에서는 *malloc(size_t) 함수를 사용해 메모리를 할당함(<stdlib.h>에 있음)
- size_t의 크기는 보통 자료형의 크기 x 인자의 개수로 지정함.
- 예: 문자열의 크기를 입력받고 문자열을 입력받아 출력하는 프로그램

```
int n;  
scanf("%d", &n);  
char *str;  
str = malloc(sizeof(char) * (n + 1));  
scanf("%s", str);  
printf("%s", str);
```



동적 할당

- `void *realloc(void *memblock, size_t size);` 함수는 이미 할당된 `memblock`을 size크기로 재할당한다.
- 배열의 크기를 늘리거나, 배열을 복사하는데 사용할 수 있다.
- `void *calloc(size_t elt_count, size_t elt_size);` 함수는 `malloc`의 인자를 분리하여, 배열을 좀 더 직관적으로 선언할 수 있도록 한다.

```
int *arr;  
// 매개변수: 인자의 개수, 인자의 크기  
// malloc(5 * sizeof(int)) 와 동일  
arr = calloc(5, sizeof(int));
```

```
// 매개변수: 원본 포인터, 배정할 크기  
// 이 경우 arr를 복제한 cpy_arr를 생성한다.  
// 해당 과정을 통한 복사는 깊은 복사이다.  
cpy_arr = realloc(arr, sizeof(arr));  
// 이 경우는 arr의 크기가 3칸 늘어난다.  
arr = realloc(arr, sizeof(arr) + 3 * sizeof(int));
```



실습 안내

- 문제 1.
- 기존에 사용하던 자료형들을 포인터 형식으로 입출력을 진행한다.
- int, float, char 순서대로 입력받고, 한 줄씩 값들을 출력한다.

```
#include <stdio.h>

int main() {
    int *int_ptr;
    float *float_ptr;
    char *char_ptr;
    // int-float-char 순서대로 입력받기

    // int-float-char 순서대로 한 줄씩 출력하기
}
```

```
input:
515 3.1415 F
output:
515 3.1415 F
```



실습 안내

- 문제 2.
- 배열의 크기만큼 숫자를 입력받고, 배열을 뒤집은 후 출력하는 프로그램을 만들어보자
- 입출력: 배열의 크기 n 과 n 개의 숫자를 입력받고, 배열을 뒤집은 후 n 개의 숫자를 출력한다.

```
#include <stdio.h>

void reverse_array(int *arr, int length);

int main() {
    int n;
    // n 입력

    int arr[n];
    // 배열 입력

    // 함수 호출
    reverse_array(arr, n);
    // output
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
}
```

input:

5

1 2 3 4 5

output:

5 4 3 2 1



실습 안내

- 문제 3.
- 학생 정보 데이터베이스를 만들어보자.
- 학생의 수를 입력받고, 각 학생의 이름, 학번, 학과를 순서대로 입력받는다.
- 학생의 이름은 최대 20자까지만 받도록 한다.
- 입력받은 데이터를 한 줄씩 이름, 학번, 학과 순서대로 출력한다.

```
/*
이런 데이터들을 묶어서 표현할 수 없을까?
다음 수업인 구조체 편에서 배워보기!
*/
char **student_name; // 학생 이름
int *student_number; // 학번
char **department; // 학과
int n; // 학생 수

// 학생 수를 입력받는다

// 입력받은 학생 수만큼 배열을 동적할당 해보자!
// 이중 포인터는 동적 할당을 두 번 해야 한다.
// 학생의 이름은 최대 20자까지만 받는다.
```

```
input:
4
Kim 2026003351 Comp.Sci
Park 20220426913 Elec
Cho 2024032177 Japanese
Julia 2025102359 Conf
output:
Kim 2026003351 Comp.Sci
Park 20220426913 Elec
Cho 2024032177 Japanese
Julia 2025102359 Conf
```



THANK YOU!

